# CodecLM: Aligning Language Models with Tailored Synthetic Data

**Zifeng Wang**[†], **Chun-Liang Li**[†], **Vincent Perot**[*], **Long T. Le**[†],
**Jin Miao**[‡], **Zizhao Zhang**[‡], **Chen-Yu Lee**[†], **Tomas Pfister**[†]
[†]Google Cloud AI Research, [‡]Google Cloud AI, [*]Google Research
{zifengw, chunliang, vperot, longtle,
jinmiao, zizhaoz, chenyulee, tpfister}@google.com

arXiv:2404.05875v1 [cs.CL] 8 Apr 2024

## Abstract

Instruction tuning has emerged as the key in aligning large language models (LLMs) with specific task instructions, thereby mitigating the discrepancy between the next-token prediction objective and users' actual goals. To reduce the labor and time cost to collect or annotate data by humans, researchers start to explore the use of LLMs to generate instruction-aligned synthetic data. Recent works focus on generating diverse instructions and applying LLM to increase instruction complexity, often neglecting downstream use cases. It remains unclear how to *tailor* high-quality data to elicit better instruction-following abilities in different target instruction distributions and LLMs. To this end, we introduce **CodecLM**, a general framework for adaptively generating high-quality synthetic data for LLM alignment with different downstream instruction distributions and LLMs. Drawing on the Encode-Decode principles, we use LLMs as codecs to guide the data generation process. We first *encode* seed instructions into metadata, which are concise keywords generated on-the-fly to capture the target instruction distribution, and then *decode* metadata to create tailored instructions. We also introduce Self-Rubrics and Contrastive Filtering during decoding to tailor data-efficient samples. Extensive experiments on four open-domain instruction following benchmarks validate the effectiveness of CodecLM over the current state-of-the-arts.

## 1 Introduction

Large language models (LLMs) have exhibited remarkable capabilities across a wide array of natural language processing (NLP) tasks (Brown et al., 2020; Ouyang et al., 2022; OpenAI, 2023a; Anil et al., 2023). In particular, LLMs can be trained for improved instruction-following through various methods, including fine-tuning on human-annotated data (Touvron et al., 2023; Bai et al., 2022) or extracted knowledge from stronger LLMs (Wang et al., 2022; Taori et al., 2023; Chiang et al., 2023; Peng et al., 2023). Recent progress in this area highlights the critical role of high-quality data in enhancing LLMs' instruction-following capabilities (Zhou et al., 2023a; Köpf et al., 2023; Chen et al., 2023b). However, acquiring such data through human annotation remains cost-prohibitive and difficult to scale, hindering further progress.

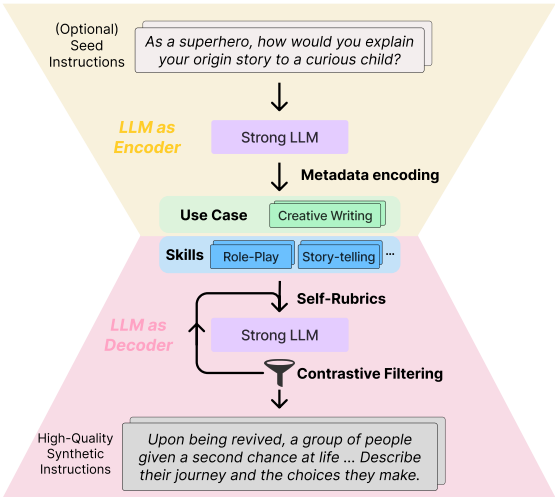


Figure 1: Overview of CodecLM. We first **encode** seed instructions into metadata to capture the underlying distribution of instructions. This metadata is then **decoded** through Self-Rubrics and Contrastive Filtering to tailor high-quality synthetic instructions that are aligned with the target instruction distribution. Intermediate instructions and responses are omitted in the figure for clarity.

As an alternative solution to human annotation, recent work explores generating instruction-response pairs for LLM alignment by prompting them with example data or prompts and iteratively refining the results (Honovich et al., 2022; Wang et al., 2022; Li et al., 2023; Xu et al., 2023). While these methods are effective at generating diverse and complex instructions for LLM alignment broadly, real-world applications often prioritize tailoring the LLM to specific downstream tasks such as individual enterprise applications or personal assistant agents (OpenAI, 2023b), which of-

ten involve different instruction distributions. This desideratum for task-specific alignment brings us to a core question for data synthesis: *how can we tailor synthetic data to align LLMs for different instruction-following tasks?*

Specifically, current data synthesis approaches fall short of providing effective solutions for task-specific LLM alignment. While prior works by Wang et al. (2022) and Xu et al. (2023) emphasize diversity and complexity as hallmarks of high-quality data, these approaches stumble when facing different downstream tasks that may involve specific instruction distributions. A diverse dataset for one task might not effectively cover the instruction distribution for another. Furthermore, the definition of "complex" instructions can be subjective and vary across tasks. To complicate matters further, an LLM might excel at some seemingly complex instructions while struggling with others that appear simple according to human-crafted criteria. These limitations underscore the need for a unified data synthesis framework that can generate tailored data to align LLMs on specific downstream tasks.

In this work, we present a novel framework, **CodecLM**, which systematically generates tailored high-quality data to align LLMs for different downstream tasks. A high-level overview of CodecLM is shown in Figure 1. Inspired by the principles of Encode-Decode process (Kramer, 1991; Kingma and Welling, 2013), we leverage a strong LLM as a codec to "encode" seed instructions from our target task into instruction *metadata* and then "decode" the metadata into tailored synthetic instructions. The metadata serves as a word-level abstraction of the input instruction distribution, including the *use case* and *skills* for effective instruction following. It can be automatically generated by encoding seed instructions, or directly provided by users with a high-level anticipation of the downstream task.

Once the metadata is extracted, we then "decode" them to generate tailored instructions. We begin by prompting a LLM with the metadata as constraints, creating basic instructions. To elevate the instruction quality, we introduce *Self-Rubrics*. It samples appropriate actions from strong LLMs to make the basic instruction more complex or challenging based on the rubrics it generates for different metadata. Intuitively, a general knowledge QA instruction about math would differ in complexity rubrics from one in creative writing about sports. With self-generated rubrics and actions based on metadata, the strong LLM crafts instructions that better align the target LLM with specific knowledge required for the downstream task. We can run Self-Rubrics iteratively to control the instruction complexity, similar to Xu et al. (2023), and finally generate the corresponding responses.

We also introduce *Contrastive Filtering* during decoding to further identify the most effective instruction-response pairs by leveraging the quality discrepancy between the target and a stronger LLM. This strategy identifies two key instruction sets: (a) those the target LLM struggles with, pushing it to improve in its weak areas for more significant gains, and (b) those the target LLM excels at, feeding them back into the Self-Rubrics process for improved data efficiency. Contrastive Filtering serves as a response-level analogy of contrastive decoding (Li et al., 2022).

CodecLM sets a new state-of-the-art on four open-domain instruction-following benchmarks with various LLM choices, demonstrating its effectiveness in LLM alignment for diverse instruction distributions.

## 2 Related Work

**Instruction Tuning for LLM Alignment.** Tuning LLM to faithfully follow instructions and align with diverse human preferences remains a significant challenge (Efrat and Levy, 2020). Early research primarily focused on cross-task generalization, where models were fine-tuned on various public NLP datasets to improve performance on diverse tasks (Raffel et al., 2020; Wei et al., 2021; Aribandi et al., 2021; Victor et al., 2022; Chung et al., 2022). More recently, researchers have extended instruction tuning to open-domains, characterized by a wider range of formats and task types. This shift has been driven by crowdsourcing human-generated instruction-response pairs (Ouyang et al., 2022; Köpf et al., 2023; Zhou et al., 2023a) and LLM-generated data (Taori et al., 2023; Chiang et al., 2023). Unlike prior work, CodecLM presents a unique approach for tailoring synthetic data to specific downstream tasks without human annotation, utilizing the concept of instruction metadata.

**Data Generation for Instruction Tuning.** To address the high cost of human annotation for high-quality instruction-response pairs, several studies advocate for automating the data generation process (Schick and Schütze, 2021; Liu et al., 2022; Meng et al., 2023). Leveraging the in-context learning (Brown et al., 2020) ability of LLMs, Wang

et al. (2022); Honovich et al. (2022) prompt LLMs with seed instructions to generate synthetic ones. These are then fed to stronger LLMs, *e.g.*, ChatGPT, to generate responses for training the target (often smaller) LLM (Taori et al., 2023). As a representative work, WizardLM (Xu et al., 2023), designs a fixed set of human-crafted operations to increase complexity of instructions and control difficulty of generated data. Zhao et al. (2023); Zhou et al. (2023a) further confirm the importance of instruction complexity for LLM alignment through empirical studies. Different from these works that rely on pre-defined rules without considering the downstream tasks, CodecLM enables automatically tailoring instructions for different downstream tasks and target LLMs. We also introduce Self-Rubrics and Contrastive Filtering to further identify the most effective instruction-response pairs.

**Distillation.** Alternatively, tuning the target LLM with responses generated from another LLM can be viewed as knowledge distillation (Hinton et al., 2015; Beyer et al., 2022). However, our focus remains on instruction generation, while still being flexible to readily integrate with existing distillation techniques (Hsieh et al., 2023; Liang et al., 2023).

Finally, we discuss some of the most relevant recent work. AttrPrompt (Yu et al., 2023) leverages LLM as attributed data generator by extracting attributes within instructions. However, it focuses solely on classification tasks and requires human intervention for attribute selection. In contrast, our work focuses on the broader context of aligning LLMs to follow open-domain instructions, eliminating the need for human efforts. MSP (Chen et al., 2023a) utilizes trainable soft prompts to control generation, but requires gradient access to the LLM. Our method, on the other hand, is readily compatible with black-box LLMs that only offer API access for high-quality data generation. SteerLM (Dong et al., 2023) analyzes quality-related aspects of responses, instead of the instructions, to capture human preference. Therefore, SteerLM can be used alongside CodecLM as a parallel approach for enhancing response quality.

## 3 Problem Statement

We study the open-domain instruction following problem (Wang et al., 2022; Taori et al., 2023; Xu et al., 2023), where instructions vary in input format and tasks. Specifically, we consider two practical scenarios: (1) Starting with a given set of $n$ seed instructions $\mathcal{D}_s = \{I_i\}_{i=1}^n$, each drawn from some underlying distribution $P_I$. For our experiments, we create a set of seed instructions using a held-out validation set. Practically, such instructions can be collected from the usage traffic of users. (2) In the absence of seed instructions, but with prior knowledge of downstream tasks, we directly start with a given set of instruction metadata $\mathcal{M}$ (see Section 4.1 for definition). The latter scenario is especially useful for end users who lack existing instruction data but wish to jumpstart LLM tailored to specific applications, similar to the concept of GPTs (OpenAI, 2023b).

We focus on the first scenario for clarity, though the second can be derived similarly by leveraging an LLM as the encoder (Section 4.1). Our goal is to generate a set of high-quality instruction-response pairs $\mathcal{D}_g = \{(I_j^{'}, R_j^{'})\}_{j=1}^m$, using a strong LLM $f_s$, and then use $\mathcal{D}_g$ to fine-tune the target LLM $f_t$. We evaluate the performance of the fine-tuned LLM $f_t$ on test instructions from the target distribution $P_I$, to which we are aligning.

## 4 CodecLM

We propose CodecLM, a general framework for generating high-quality instruction-response pairs tailored to different downstream tasks and LLMs, eliminating the need for human annotation. See Figure 2 for method overview.

### 4.1 LLM as Codec for Instructions

In this section, we introduce the concept of using a strong LLM as a codec, *i.e.*, both encoder and decoder, for instruction generation.

**LLM as Encoder with Instruction Metadata.** We begin by encoding the given seed instructions $\mathcal{D}_s = \{I_i\}_{i=1}^n$ into instruction *metadata* $\mathcal{M}$, *i.e.*, keywords that capture the underlying target instruction distribution. Inspired by the task pool by Wang et al. (2022) and the post-hoc analysis on skill distribution by Xu et al. (2023), we define the metadata as encompassing two key aspects: *use case* and *skills*. Use case describes the intended task (*e.g.*, question answering or creative writing), while Skills are the knowledge the LLM required to have to successfully respond to the given instruction (*e.g.*, algorithms or communication). Skills are often generalizable to different use cases. Therefore, each instruction has a single use case and may involve multiple skills. To extract this metadata, we leverage the strong LLM $f_s$ following

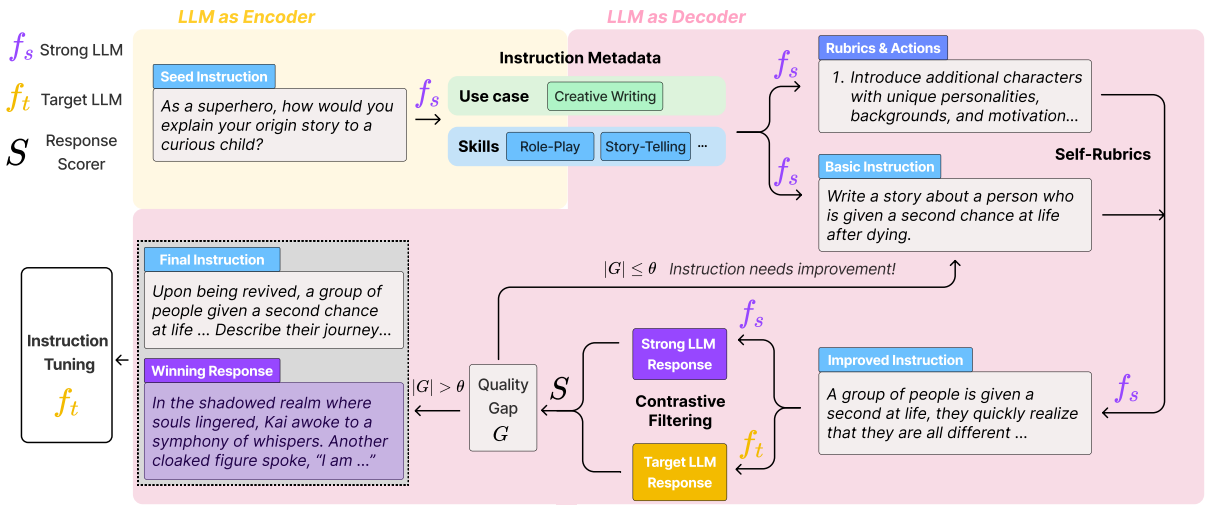


Figure 2: Overview of the proposed CodecLM. First, the strong LLM $f_s$ encodes the seed instruction into instruction metadata, specifying its use case and skills required for responses. Next, $f_s$ decodes metadata into basic instructions. Meanwhile, Self-Rubrics leverages $f_s$ to generate rubrics and actions to improve the basic instruction, tailoring them for the downstream task. Finally, Contrastive Filtering uses a scoring function $S$ to compares $f_s$ and $f_t$'s responses. The most effective pairs are selected for aligning the LLM, while less effective instructions are sent for further improvement. In this figure, the strong LLM's response is winning against the target one's, so we select the corresponding pair for instruction tuning the target LLM.

the prompt template in Figure 7, Appendix A.9. While richer definitions are possible based on finer-grained instruction-following metrics (Zhou et al., 2023b), we prioritize use case and skills for their broad applicability across diverse instruction distributions. Future work can explore extending this metadata further.

For each instruction $I_i$, we extract the corresponding use case $u_i$ and set of skills $\boldsymbol{s}_i$. We then have the set of metadata as $\mathcal{M} = \{(u_i, \boldsymbol{s}_i)\}_{i=1}^{n}$. Instructions may share or partially overlap in their $u_i$'s and $\boldsymbol{s}_i$, reflecting the distribution of tasks and capabilities within the seed instructions. Use cases and skills are generated on-the-fly, not limited to some predefined sets, enabling broader applicability. However, we can always provide such constraints with our prior knowledge, or even directly write out metadata without any seed instructions.

**LLM as Decoder for Instruction Generation.** Given the metadata $\mathcal{M}$, we decode metadata into synthetic instructions, following a generation and tailoring paradigm. For each use case and skills pair in $\mathcal{M}$, we list them as constraints to prompt the strong LLM $f_s$ to generate multiple instructions. Therefore, the generated instructions are for the given use case, and require the given skills to be responded. Moreover, to prevent the LLM from generating repetitive instructions, we encourage its generation to be diverse in the prompt, and do not provide any demonstrations that the LLM might copy from. The example prompt template for generating basic instructions is in Figure 8, Appendix A.9. Continuing the decoding process, we then tailor the basic instructions for more effective alignment through Self-Rubrics (Section 4.2) and Contrastive Filtering (Section 4.3).

## 4.2 Instruction Tailoring via Self-Rubrics

Metadata-conditioned instructions lay the groundwork for aligning the target LLM to desired tasks. Studies suggest that more complex instructions can improve alignment performance (Xu et al., 2023; Zhao et al., 2023). A common practice is to involve human experts crafting general guidance to complicate instructions, such as adding reasoning steps or constraints. However, this one-size-fits-all strategy falls short for diverse instructions. Tailoring guidance to different tasks, like solving calculus problems versus writing news articles, requires distinct approaches.

Therefore, we introduce Self-Rubrics, which leverages the strong LLM to tailor instructions by adjusting their complexity according to the extracted metadata. Self-Rubrics first guides the LLM to generate metadata-specific rubrics for assessing instruction complexity. Then, informed by these rubrics, the LLM generates a corresponding set of actions to enhance the instruction's complexity. For metadata $(u_i, \boldsymbol{s}_i)$, the corresponding set of generated actions is $\boldsymbol{a}_i$. Our generated actions are more domain-specific, and unambiguous than generic rules crafted by human, making the complicated

instructions better tailored towards the target distribution captured by the metadata. For example, for the use case of "business plan development" and skills of "market research and planning", generic rules like "add reasoning steps" is vague and inappropriate. On the contrary, Self-Rubrics is able to generate actions like "add SWOT analyisis" and "include comparison with market competitors" (see Appendix A.8 for the full details) to complicate the instruction. The prompt template to generate rubrics and actions for instruction improvement is shown in Figure 9, Appendix A.9.

With the obtained actions $\{\boldsymbol{a}_i\}_{i=1}^{n}$, we can iteratively prompt $f_s$ to complicate the basic instructions, following the prompt template in Figure 10. We randomly sample an action $\boldsymbol{a}_i$ from the multiple actions generated for a pair of use case and skills. This design choice not only enables controlled complexity (Xu et al., 2023), but also prevents potential confusion between different actions for the LLM.

### 4.3 Instruction Selection via Contrastive Filtering

While Self-Rubrics tailors complex instructions based on instruction metadata, not all instructions are equally effective for instruction tuning, regardless of their complexity (Chen et al., 2023b; Zhou et al., 2023a). Intuitively, exposing the target LLM to instructions it finds challenging can effectively identify its areas for improvement. Therefore, it is crucial to select the most impactful instructions for aligning the target LLM.

We therefore introduce Contrastive Filtering, a method to select the instructions that can effectively enhance the target LLM $f_t$. For clarity, we define the space of all natural language sequences as $\mathcal{N}$. We have the strong LLM $f_s : \mathcal{N} \rightarrow \mathcal{N}$, the target LLM $f_t : \mathcal{N} \rightarrow \mathcal{N}$, and a scoring function $S : \mathcal{N} \rightarrow \mathbb{R}$ to evaluate response quality. In practice, $S$ is obtained by reusing the strong LLM $f_s$ with a prompt template (Figure 11, Appendix A.9) adapted from the Vicuna pairwise evaluation template (Taori et al., 2023; Chiang et al., 2023). To mitigate potential position bias, we average the scores obtained by exchanging the positions of two responses (Chiang et al., 2023). We observe using $f_s$ for scoring works quite well in practice, so we prioritize this option for simplicity. Given an input instruction $I \in \mathcal{N}$, we obtain responses from both LLMs as $f_s(I)$ and $f_t(I)$, respectively. We then define the *quality gap* $G : \mathcal{N} \rightarrow \mathbb{R}$ between these responses to estimate the *effectiveness* of the instruction: $G(I) = S(f_s(I)) - S(f_t(I))$.

The quality gap metric $G$ reflects how much the target LLM benefits from the strong LLM for each instruction $I$. As demonstrated in Figure 2, here are two possible cases: (1) $|G(I)| > \theta$, where $\theta \in \mathbb{R}$ is a certain threshold. This indicates that: Either the strong LLM has a much better response than the target LLM, we add $(I, f_s(I))$ to our high-quality instruction-response pool $\mathcal{D}_g$ to fill the gap; Or rarely, the target LLM gives much better response than the strong LLM, we add $(I, f_t(I))$ to $\mathcal{D}_g$ as as an implicit regularization to keep the target LLM's desirable behavior to certain instructions. (2) $|G(I)| \leq \theta$, where the quality of responses from both LLMs is similar, so learning from $I$ does not lead to much gain. We then send $I$ to the next Self-Rubrics iteration for further improvement.

Contrastive Filtering complements Self-Rubrics to select effective instruction-response pairs by calibrating the target LLM's instruction-following capability with the strong LLM's. Analogous to Contrastive Decoding (Li et al., 2022) at response-level, Contrastive Filtering can also be regarded as LLM-feedback (Madaan et al., 2023) with the interaction of two LLMs. While we adopt the strong LLM as scoring function to measure the quality gap, our framework can be compatible with and potentially benefit from the advances in more reliable and comprehensive scoring and feedback systems (Lee et al., 2023), and we leave it as promising future work.

## 5 Experiments

We conduct comprehensive experiments to evaluate CodecLM using different LLMs on multiple representative benchmarks, closely following well-established evaluation settings for open-domain instruction following in prior work (Xu et al., 2023; Chen et al., 2023b). We also conduct a case study in Appendix A.8 to illustrate how CodecLM tailors an instruction step by step.

### 5.1 Evaluation Benchmarks

We evaluate CodecLM on four widely-used open-domain instruction-following benchmarks with diverse instruction distributions to reduce evaluation bias. Our test benchmarks include Evol-Instruct (Xu et al., 2023), Vicuna (Chiang et al., 2023), Self-Instruct (Wang et al., 2022) and Koala (Geng et al., 2023). To complement the evaluation, we also evaluate on two standard NLP

benchmarks MMLU (Hendrycks et al., 2020) and BBH (Suzgun et al., 2022) in Appendix A.7. Please refer to Appendix A.1 for benchmark details.

### 5.2 Baseline Methods

We compare our method against state-of-the-art data generation approaches for instruction tuning. For fair comparison, we provide all methods the same LLM backbones when possible. Moreover, we control the number of instruction-response pairs the same for all methods to ablate the effect of data quantity. Baseline methods include **Self-Instruct** (Wang et al., 2022), **Alpagasus** (Chen et al., 2023b), **Tree-Instruct**, **WizardLM** (Xu et al., 2023), and **WizardLM+**, an enhanced version of WizardLM using the same basic instructions generated from CodecLM as seed instructions. Baseline details are presented in Appendix A.2.

### 5.3 Experiment and Evaluation Details

**LLM Backbones.** We adopt LLaMA-based (Touvron et al., 2023) and PaLM-based (Anil et al., 2023) LLMs as our target LLMs in our experiments. For LLaMA-based target LLMs, we use Gemini-Pro (Team et al., 2023) as the strong LLM, and LLaMA-7B, -13B as the target LLMs. For PaLM-based target LLMs, we use text-unicorn as the strong LLM, and text-bison as the target LLM. PaLM-based models and Gemini-Pro are accessible through Google Cloud API[1].

**Implementation Details of CodecLM.** We split all benchmarks into 20% validation set and 80% evaluation set. We extract the instruction metadata from the validation set, see Appendix A.3 for more details. Depending on the specified total data size, we prompt the strong LLM to generate equal number of base instruction per metadata. We generate 500-8000 synthetic data throughout the experiments. We generate 4 rubrics and corresponding actions. At each iteration, we randomly choose 1 action for improving instruction. We run Self-Rubrics at most 4 iterations. For Contrastive Filtering, We set the scoring scale to 10 and the filtering threshold to 3 for all experiments. We align these configurations with Xu et al. (2023) and leave more detailed rationales of these configurations, additional hyperparameter settings, and training details in Appendix A.3-A.4.

**Evaluation.** Assessing how well LLMs follow instructions is complex, arising from the fact that an instruction has various valid responses, and the challenge of replicating human evaluation. Recent advances in automatic evaluation on instruction following (Dubois et al., 2023; Zheng et al., 2023) demonstrate that LLM-based evaluators are scalable, explainable, and consistent with human evaluations. Therefore, we adopt widely-used Vicuna pairwise evaluator (Chiang et al., 2023) based on ChatGPT to compare the response quality from two LLMs for its accessibility in price and efficiency. The evaluation prompt template is in Figure 12, Appendix A.9. We include GPT-4 based evaluation results in Appendix A.6 to demonstrate the consistency of LLM-based evaluators. To mitigate position bias that the LLM evaluator may have, we conduct every evaluation twice by exchanging response orders. A response is considered better only if it wins twice. Following (Chen et al., 2023b), we set the temperature to 0.0 to reduce evaluation randomness, and left other parameters as default.

Similar to prior work (Xu et al., 2023; Zhao et al., 2023), we compute the total ratio of wins and ties of a target LLM against the strong LLM, to indicate how much model capacity the target LLM recovers from the strong LLM (often treated as the upper bound performer). CRR simplifies the combinatorial pairwise comparisons between all target LLMs. We name the metric as *Capacity Recovery Ratio* (CRR), where $\texttt{CRR} = \frac{\texttt{wins+ties}}{\texttt{total comparisons}}$. In experiments, we observe that the number of ties often dominates the number of wins, since the strong LLM is much capable than the target model. So we do not put additional weights on wins in the calculation. To demonstrate CRR faithfully reflects model performance, we show the exact number of wins, ties and losses in Appendix A.5 on Evol-Instruct. We would like to emphasize our focus on the gap in CRR between different methods instead of the absolute value, since the absolute value may based on the specific LLM evaluator we choose.

### 5.4 Open-Domain Instruction Following

**Results with LLaMA-based Target LLMs.** Table 1 summarizes the performance of CodecLM and the comparing baselines with 2000 synthetic data for instruction tuning. All methods are trained on LLaMA-7B or -13B as the target LLM and compared against Gemini-Pro, the strong LLM that generates the data. CodecLM outperforms comparing methods consistently on all benchmarks, with two target LLMs of different sizes. The consistently superior performance of CodecLM highlights its

[1] https://cloud.google.com/vertex-ai

Table 1: Results with LLaMA-based target models on four open-domain instruction following benchmarks. Each method trains a target model based on LLaMA-7B or -13B, and compares against the strong model, Gemini-Pro. The reported metric Capacity Recovery Ratio (%), $\texttt{CRR} = \frac{\texttt{wins+ties}}{\texttt{total comparisons}}$. Larger CRR means better performance.

| Methods | LLaMA-7B vs. Gemini-Pro | | | | LLaMA-13B vs. Gemini-Pro | | | |
|---|---|---|---|---|---|---|---|---|
| | **Evol-Ins.** | **Vicuna** | **Koala** | **Self-Ins.** | **Evol-Ins.** | **Vicuna** | **Koala** | **Self-Ins.** |
| Self-Instruct | 72.02 | 81.25 | 67.78 | 65.87 | 75.69 | 86.25 | 77.22 | 69.05 |
| Alpagasus | 75.23 (+3.2) | 81.25 (+0.0) | 71.11 (+3.3) | 70.24 (+4.4) | 79.82 (+4.1) | 87.50 (+1.3) | 77.78 (+0.6) | 71.03 (+2.0) |
| Tree-Instruct | 75.23 (+3.2) | 81.25 (+0.0) | 72.78 (+5.0) | 68.65 (+2.8) | 82.57 (+6.9) | 87.50 (+1.3) | 80.56 (+3.3) | 79.37 (+10.3) |
| WizardLM | 74.31 (+2.3) | 76.25 (-5.0) | 65.56 (-2.2) | 71.43 (+5.6) | 82.11 (+6.4) | 86.25 (+0.0) | 78.89 (+1.7) | 76.19 (+7.1) |
| WizardLM+ | 75.69 (+3.7) | 83.75 (+2.5) | 68.33 (+0.6) | 72.22 (+6.4) | 84.40 (+8.7) | 88.75 (+2.5) | 81.11 (+3.9) | 79.76 (+10.7) |
| CodecLM (ours) | **79.82 (+7.8)** | **88.75 (+7.5)** | **74.44 (+6.7)** | **78.17 (+12.3)** | **86.70 (+11.0)** | **90.00 (+3.8)** | **82.22 (+5.0)** | **83.33 (+14.3)** |

Table 2: CRR Results on PaLM-based models. Each method trains a target model based on text-bison, and compares against the strong model, text-unicorn.

| Methods | text-bison vs. text-unicorn | | | |
|---|---|---|---|---|
| | **Evol-Ins.** | **Vicuna** | **Self-Ins.** | **Koala** |
| text-bison | 87.16 | 81.25 | **74.21** | 77.47 |
| Alpagasus | 82.11(-5.1) | 81.25 (+0.0) | 67.86 (-6.4) | 73.33 (-4.1) |
| WizardLM+ | 84.40 (-2.8) | 78.75 (-2.5) | 69.44 (-4.8) | 73.89 (-3.6) |
| CodecLM (ours) | **88.53 (+1.4)** | **86.25 (+5.0)** | 72.22 (-2.0) | **80.56 (+3.1)** |

generalizability to different downstream instruction distributions and target LLMs. Both Tree-Instruct and variants of WizardLM focus on the importance of instruction complexity, however, their performances are not always better than Alpagasus with simple instructions, especially with larger target LLM. This observation indicates that the effectiveness of data cannot be solely determined by instruction complexity, and validates the motivation of our design of Self-Rubrics and Contrastive Filtering. Moreover, the win of WizardLM+ over WizardLM confirms the efficacy of instruction distribution matching via instruction metadata. When shifting the target LLM from LLaMA-7B to -13B, all methods get a significant performance boost, which accords with prior discoveries on scaling model size (Wei et al., 2021).

**Results with PaLM-based Models.** Table 2 summarizes the results of CodecLM and the best performing baselines in LLaMA-based experiments. We generate 1000 synthetic data due to computation budget. Since text-bison is a proprietary model that has been aligned with various techniques including instruction tuning, we also include it as a baseline approach. Interestingly, text-bison obtains strong performance across different benchmarks. Both Alpagasus and WizardLM+ underperform text-bison, suggesting it is non-trivial to improve upon a well-tuned LLM continually. CodecLM, on the contrary, outperforms text-bison in most cases, thanks to our core designs that adaptively tailor high quality data pairs to improve the target LLM.

Table 3: Ablation study of CodecLM's core designs. All components contribute to the final performance.

| Metadata | Self-Rubrics | Contrastive Filtering | CRR |
|---|---|---|---|
| ✗ | ✗ | ✗ | 72.02 |
| ✓ | ✗ | ✗ | 75.23 |
| ✓ | ✓ | ✗ | 77.52 |
| ✓ | ✓ | ✓ | 79.82 |

## 5.5 Ablation Study

In this section, we conduct comprehensive ablation studies to empirically explore the effectiveness of CodecLM. We mainly conduct experiments with LLaMA-7B model as the target LLM, Gemini-Pro as the strong LLM, and report the CRR on the Evol-Instruct benchmark.

**Effectiveness of Core Designs.** We show component-wise contributions in our framework in Table 3. The 1st row has the result from Self-Instruct as a baseline; In the 2nd row, we only align the LLM with basic instructions from instruction metadata; We gradually add Self-Rubrics and Contrastive Filtering in the 3rd and 4th rows, respectively. We clearly observe that every component contributes to the final performance. Interesting, the performance of using basic instructions from metadata is even on par with that of WizardLM+ in Table 1. This observation indicates that human-crafted strategies for complicating instructions may not fit different types of instructions. On the contrary, Self-Rubrics adaptively generates instruction improving actions based on different metadata, resulting in better tailored instructions for the target LLM. Further improvements from Contrastive Filtering demonstrate that selected data are indeed more effective for alignment.

**Effect of Number of Iterations.** We demonstrate the effect of number of CodecLM iterations in Figure 3. In particular, we count the proportion of data from each iteration in all synthesized data $\mathcal{D}_g$ and show it in the blue bar chart with left y-axis. We also draw the target model performance in CRR after training on the synthetic data up un-

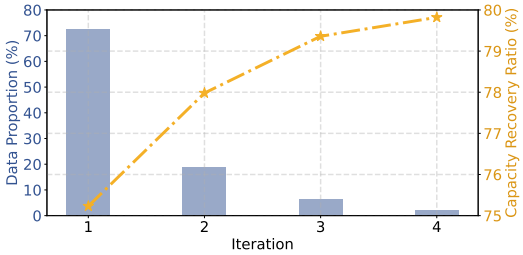


Figure 3: Data proportion from each iteration and the corresponding CRR performance at each iteration.

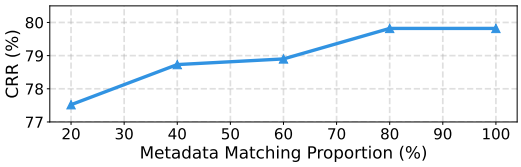


Figure 4: Metadata matching proportion vs. CRR.

til the current iteration in the yellow line chart with right y-axis. From the data proportion bar chart, we observe that more than 70% of the data comes from the first iteration. This indicates Contrastive Filtering successfully collects less complex yet challenging instructions, which are critical for building up the instruction-following ability of the target LLM. Starting from the second iteration, the data proportion gets increasingly small. However, similar to the *less is more for alignment* observation (Zhou et al., 2023a), high-quality and more complex instructions indeed contribute to the final performance despite less in quantity.

**Exploration on Distribution Matching.** As shown by previous results, generating metadata extracted from the downstream instruction distribution indeed helps. However, in practice, the extracted or human-written metadata may not be able to precisely characterize the instruction distribution. Therefore, it is necessary to explore the performance of CodecLM when the distribution represented by instruction metadata does not fully match the test distribution. As the true test distribution is complicated and not known as a prior, we approximate various extent of distribution matching by random subsampling from the set of metadata $\mathcal{M}$. To control the effect of data quantity, we keep the total number of instruction-response pairs the same for each case. For example, when subsampling 20% of $\mathcal{M}$, we prompt the strong LLM to generate 5 times more instructions for each metadata accordingly. The result is shown in the upper part of Figure 4, and we did observe the trend that the better instruction metadata captures the underlying distribution, the better performance the target LLM can achieve. Moreover, when the metadata matching proportion is equal or greater than 60%, we obtain close performance as the fully-matched result. This observation highlights CodecLM's robustness under potential instruction metadata mismatch.

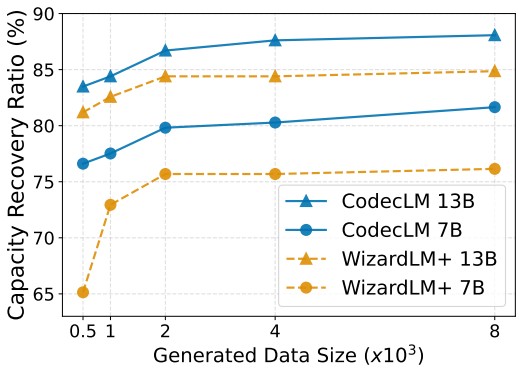


Figure 5: Scaling with model size and data quantity.

**Scaling with Model Size and Data Quantity.** To explore how our method scales with different synthetic data quantities and model sizes, we conduct experiments by comparing CodecLM with WizardLM+, the most competitive baseline. The experiment results on Evol-Instruct with LLaMA-7B and -13B as the target LLM are presented in Figure 5. Both methods get increasingly better performance with more synthetic data and larger target models. CodecLM consistently outperforms WizardLM+ under all cases, demonstrating its great data efficiency and scalability. We expect the gain will gradually diminish after we generate more than 8k synthetic data, due to the intrinsic ability gap between the target models and the strong LLM.

## 6 Conclusion

In this work, we propose CodecLM to tailor synthetic data for LLM alignment with different target instruction distributions and LLMs. We show that CodecLM effectively captures the underlying instruction distribution via instruction metadata, and further tailor the most effective instruction-response pairs through Self-Rubrics and Contrastive Filtering. CodecLM provides a potent solution towards adapting LLMs for customized uses, without the necessity of human annotation. We believe CodecLM serves as a general framework for targeted LLM alignment, which opens the door to multiple promising research directions within the framework, such as richer metadata definition, better prompt design, and more reliable LLM-based scorer. CodecLM can also benefit from orthogonal research fields, and we continue the discussion in Ethical Considerations and Limitations sections.

## Ethical Considerations

Although CodecLM serves as an effective data synthesis framework for LLM alignment, we should also reflect on the ethical impact of our work. Our method leverages LLMs to generate instruction-response pairs. Similar to human annotators who might make unconscious mistakes during the data annotation process, LLMs also sometimes generate unethical, toxic or misleading instructions and responses (Bender et al., 2021). Moreover, as we train a target LLM using the generated data, the resulting instruction-tuned LLM might also carry the bias and fairness issues (Gallegos et al., 2023) from the original model. Although we conducted manual inspection as specified in Appendix A.3, in practice, we should adopt existing techniques (Hanu and Unitary team, 2020; Thakur et al., 2023) to detoxify and mitigate bias from LLMs used in CodecLM, and design more strict inspection and filtering rules to clean up the generated data. Due to the flexibility of our framework, we envision future progress in the domain of reducing bias and fairness issues can be complementary to CodecLM.

## Limitations

We acknowledge the limitations of CodecLM from the following aspects to inspire future research opportunities in the field of LLM alignment.

First of all, as discussed in the Ethical Considerations, our method requires a strong LLM to generate the data, so the performance of our method depends on the quality of the LLM and may inherit bias and fairness issues from it. On the other hand, CodecLM can benefit from stronger LLMs improved with advanced bias-reducing and fairness-enhancing approaches.

Secondly, as an orthogonal direction, our method did not explore robustness of the instruction-tuned model towards adversarial attacks such as prompt injection (Liu et al., 2023) and jailbreaking (Zou et al., 2023). In practice, we should apply adversarial defense techniques (Jain et al., 2023) accordingly to the instruction-tuned LLM from our method.

Moreover, we mainly use LLM-based automatic evaluation methods following recent works in data synthesis for alignment. Although recent studies (Chiang et al., 2023; Dubois et al., 2023) demonstrate LLM-based evaluation is largely consistent with human evaluation, the scalability and reliability of LLM-based evaluators still have room for improvements. Although we include some standard benchmark results in Appendix A.7 to complement LLM-based evaluation results, we still believe the progress in better evaluating LLMs can lead to a more reliable demonstration of the effectiveness of our method.

Finally, as shown in Section 5.5, although CodecLM is robust to moderate distribution mismatch, its performance still depends on how well the metadata captures the underlying instruction distribution. In practice, our collected seed instruction might differ from the actual test instructions. Or in the case that we directly create metadata from user specification, the users might change their mind at test time to send the model out-of-distribution instructions beyond the original metadata. As a consequence, CodecLM may suffer performance degradation under distribution mismatch. As a remedy, we can constantly collect user instruction traffic or user feedback to update the generated data from CodecLM, and continuously update the target LLM.

We hope future work can leverage CodecLM as a flexible data synthesis framework for LLM alignment, so that advances in the field can be integrated into CodecLM to reduce its current limitations.

## References


Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.

Vamsi Aribandi, Yi Tay, Tal Schuster, Jinfeng Rao, Huaixiu Steven Zheng, Sanket Vaibhav Mehta, Honglei Zhuang, Vinh Q Tran, Dara Bahri, Jianmo Ni, et al. 2021. Ext5: Towards extreme multi-task scaling for transfer learning. *arXiv preprint arXiv:2111.10952*.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*.

Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623.

Lucas Beyer, Xiaohua Zhai, Amélie Royer, Larisa Markeeva, Rohan Anil, and Alexander Kolesnikov. 2022.

Knowledge distillation: A good teacher is patient and consistent. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10925–10934.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Derek Chen, Celine Lee, Yunan Lu, Domenic Rosati, and Zhou Yu. 2023a. Mixture of soft prompts for controllable data generation. *arXiv preprint arXiv:2303.01580*.

Lichang Chen, Shiyang Li, Jun Yan, Hai Wang, Kalpa Gunaratna, Vikas Yadav, Zheng Tang, Vijay Srinivasan, Tianyi Zhou, Heng Huang, et al. 2023b. Alpagasus: Training a better alpaca with fewer data. *arXiv preprint arXiv:2307.08701*.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2022. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.

Yi Dong, Zhilin Wang, Makesh Narsimhan Sreedhar, Xianchao Wu, and Oleksii Kuchaiev. 2023. Steerlm: Attribute conditioned sft as an (user-steerable) alternative to rlhf. *arXiv preprint arXiv:2310.05344*.

Yann Dubois, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpacafarm: A simulation framework for methods that learn from human feedback. *arXiv preprint arXiv:2305.14387*.

Avia Efrat and Omer Levy. 2020. The turking test: Can language models understand instructions? *arXiv preprint arXiv:2010.11982*.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2023. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*.

Isabel O Gallegos, Ryan A Rossi, Joe Barrow, Md Mehrab Tanjim, Sungchul Kim, Franck Dernoncourt, Tong Yu, Ruiyi Zhang, and Nesreen K Ahmed. 2023. Bias and fairness in large language models: A survey. *arXiv preprint arXiv:2309.00770*.

Xinyang Geng, Arnav Gudibande, Hao Liu, Eric Wallace, Pieter Abbeel, Sergey Levine, and Dawn Song. 2023. Koala: A dialogue model for academic research. Blog post.

Laura Hanu and Unitary team. 2020. Detoxify. Github. https://github.com/unitaryai/detoxify.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick. 2022. Unnatural instructions: Tuning language models with (almost) no human labor. *arXiv preprint arXiv:2212.09689*.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. *arXiv preprint arXiv:2305.02301*.

Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614*.

Diederik P Kingma and Max Welling. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi-Rui Tam, Keith Stevens, Abdullah Barhoum, Nguyen Minh Duc, Oliver Stanley, Richárd Nagyfi, et al. 2023. Openassistant conversations–democratizing large language model alignment. *arXiv preprint arXiv:2304.07327*.

Mark A Kramer. 1991. Nonlinear principal component analysis using autoassociative neural networks. *AIChE journal*, 37(2):233–243.

Gyeong-Geon Lee, Ehsan Latif, Xuansheng Wu, Ninghao Liu, and Xiaoming Zhai. 2023. Applying large language models and chain-of-thought for automatic scoring. *arXiv preprint arXiv:2312.03748*.

Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Luke Zettlemoyer, Omer Levy, Jason Weston, and Mike Lewis. 2023. Self-alignment with instruction backtranslation. *arXiv preprint arXiv:2308.06259*.

Xiang Lisa Li, Ari Holtzman, Daniel Fried, Percy Liang, Jason Eisner, Tatsunori Hashimoto, Luke Zettlemoyer, and Mike Lewis. 2022. Contrastive decoding: Open-ended text generation as optimization. *arXiv preprint arXiv:2210.15097*.

Chen Liang, Simiao Zuo, Qingru Zhang, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2023. Less is more: Task-aware layer-wise distillation for language model compression. In *International Conference on Machine Learning*, pages 20852–20867. PMLR.

Alisa Liu, Swabha Swayamdipta, Noah A Smith, and Yejin Choi. 2022. Wanli: Worker and ai collaboration for natural language inference dataset creation. *arXiv preprint arXiv:2201.05955*.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2023. Prompt injection attack against llm-integrated applications. *arXiv preprint arXiv:2306.05499*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*.

Yu Meng, Martin Michalski, Jiaxin Huang, Yu Zhang, Tarek Abdelzaher, and Jiawei Han. 2023. Tuning language models as training data generators for augmentation-enhanced few-shot learning. In *International Conference on Machine Learning*, pages 24457–24477. PMLR.

OpenAI. 2023a. Gpt-4 technical report. *ArXiv*, abs/2303.08774.

OpenAI. 2023b. Introducing gpts. https://openai.com/blog/introducing-gpts.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. 2023. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551.

Timo Schick and Hinrich Schütze. 2021. Generating datasets with pretrained language models. *arXiv preprint arXiv:2104.07540*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, et al. 2022. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

Himanshu Thakur, Atishay Jain, Praneetha Vaddamanu, Paul Pu Liang, and Louis-Philippe Morency. 2023. Language models get a gender makeover: Mitigating gender bias with few-shot data interventions. *arXiv preprint arXiv:2306.04597*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Sanh Victor, Webson Albert, Raffel Colin, Bach Stephen, Sutawika Lintang, Alyafeai Zaid, Chaffin Antoine, Stiegler Arnaud, Raja Arun, Dey Manan, et al. 2022. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*.

Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, David Wadden, Kelsey MacMillan, Noah A Smith, Iz Beltagy, et al. 2023. How far can camels go? exploring the state of instruction tuning on open resources. *arXiv preprint arXiv:2306.04751*.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-instruct: Aligning language model with self generated instructions. *arXiv preprint arXiv:2212.10560*.

Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2023.

Large language models as optimizers. *arXiv preprint arXiv:2309.03409*.

Yue Yu, Yuchen Zhuang, Jieyu Zhang, Yu Meng, Alexander Ratner, Ranjay Krishna, Jiaming Shen, and Chao Zhang. 2023. Large language model as attributed training data generator: A tale of diversity and bias. *arXiv preprint arXiv:2306.15895*.

Yingxiu Zhao, Bowen Yu, Binyuan Hui, Haiyang Yu, Fei Huang, Yongbin Li, and Nevin L Zhang. 2023. A preliminary study of the intrinsic relationship between complexity and alignment. *arXiv preprint arXiv:2308.05696*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. 2023a. Lima: Less is more for alignment. *arXiv preprint arXiv:2305.11206*.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023b. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*.

Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models.

# A Appendix

## A.1 Benchmark Details

The details of the open-instruction following benchmarks are included below:

- Evol-Instruct (Xu et al., 2023) includes 218 real-world human instructions from diverse sources such as online open-source projects, platforms, and forums.
- Vicuna (Chiang et al., 2023) includes 80 diverse instructions generated by GPT-4 through prompt engineering.
- Self-Instruct (Wang et al., 2022) includes 252 expert-written instructions motivated by user-oriented applications.
- Koala (Geng et al., 2023) includes 180 conversation-style real user instructions that were posted online.

All these benchmarks consist of English instructions from multiple categories or tasks. However, though sharing some common use cases such as general knowledge QA and coding, the coverage of the instructions in different benchmarks are indeed different. For example, Xu et al. (2023) discuss in detail how Evol-Instruct is different from Vicuna in instruction distribution. The difference between instruction distributions effectively mimic the practical scenario where we have different downstream tasks.

The details of the additional standard NLP benchmarks are included below:

- MMLU (Hendrycks et al., 2020), Massive Multitask Language Understanding, is a benchmark designed to measure capability of language models. It covers 57 subjects across STEM, the humanities, the social sciences, and more areas. We only use the test split for reporting the test results, and report the average score across all tasks.
- BBH (Suzgun et al., 2022), BIG-Bench-Hard, includes 23 challenging BIG-Bench tasks that prior language models did not outperform average human-raters.

All benchmarks are publicly available for non-commercial research purposes, and we strictly limit their usage in this research work. We also carefully check these datasets and make sure that no personal information is involved.

## A.2 Baseline Details

**Self-Instruct** (Wang et al., 2022) generates instructions by prompting LLM with existing seed instructions as few-shot demonstrations. Here we randomly subsample the Alpaca (Taori et al., 2023) dataset as seed instructions. Since Alpaca itself is based on Self-Instruct, using its subset as seed is a natural continuation of the Self-Instruct method.

**Alpagasus** (Chen et al., 2023b) selectively filters data using ChatGPT-based response quality evaluator. Closely following the original approach, we adopt the strategy upon instruction-response pairs generated by Self-Instruct.

**Tree-Instruct** (Zhao et al., 2023) improves instruction quality by prompting the LLM to implicitly complicate instruction through its semantic tree. Following the original paper, we use the subsampled Alpaca dataset as seed data. We set the number of tree nodes to 10 for best possible performance.

**WizardLM** (Xu et al., 2023) iteratively complicates instructions by prompting the LLM with a set of pre-defined evolution operations. Given the popularity and effectiveness of WizardLM, we experiment it with two variants: the original version using Alpaca as seed data, and the enhanced version uses the same set of basic instructions generated from CodecLM as seed data. We name the later variant as **WizardLM+** as its enhanced by components of our framework.

## A.3 Additional Implementation Details

We augment the metadata to 200 by mix-and-matching use cases and skills from different instructions. We randomly sample one use case from $\{u_i\}_{i=1}^n$, and pair it with one or more skills sampled without replacement from $\bigcup_{i=1}^n \boldsymbol{s}_i$. Although most skills are generalizable between use cases, we still conduct manual sanity check to exclude unreasonable use case and skills pairs. We align our hyper-parameters for iteratively improving instructions via Self-Rubrics with prior work (Xu et al., 2023): We generate 4 rubrics and corresponding actions, and at each iteration, we randomly choose 1 action for improving instruction. For fair comparison with WizardLM, we also use at most 4 improve iterations for each instruction (we count basic prompt generation as the first iteration). For Contrastive Filtering, we always use the strong LLM itself as the scorer. We set the scoring scale to 10 and the filtering threshold to 3 for all experiments. We obtain the threshold by developing on the AlpacaE-

val (Dubois et al., 2023) dataset. And we find this threshold works generally well across different settings. Moreover, for LLaMA-based models, using their Alpaca (Taori et al., 2023) counterparts as the target LLM for response generation in Contrastive Filtering works better than the original model that is not instruction tuned. For metadata extraction, base instruction generation and Self-Rubrics, we use a inference temperature of 0.7. We set the maximum number of tokens for generation to 2048 for LLaMA-based models, and 1024 for PaLM-based models due to API constraints. Moreover, although we set aside 20% validation set for metadata extraction, we still report the performance on the full test set in the main paper, the reasons are as follows: (1) We observe removing the validation set from the full test benchmark will not change the relative superior performance of our method, the performance gap between our method and baselines remains almost the same. Therefore, we keep them in for better reproducibility. (2) By carefully checking the generated instructions, we notice that none of the generated instructions overlap with the original validation instructions, so no data leaking happens during the data generation process.

We conduct manual inspection on the generated data to make sure no personal information or offensive contents are generated.

### A.4 Training Details

For LLaMA-based models, we follow the practices in instruction tuning in prior works (Zhou et al., 2023a; Chen et al., 2023b). We use AdamW optimizer with $\beta_1 = 0.9, \beta_2 = 0.95$ to finetune the target model for 15 epochs, as suggested by Zhou et al. (2023a) for smaller data size. We set the initial learning rate to $1 \times 10^{-5}$ and linearly decaying to $1 \times 10^{-6}$ by the end of training. We set per GPU batch size to 8, which is equivalent to a total batch size of 64, as we use 8 A100 GPUs for training. The maximum token length is set to 2048.

For PaLM-based models, we follow the default instruction tuning setting on Google Cloud's LLM tuning web UI. We set the number of tuning steps to 2000, the learning rate multiplier to 1, and use the TPU training option.

### A.5 Detailed Comparison Results

We show the details of pairwise comparison on Evol-Instruct benchmark with LLaMA-based models, as a demonstration of how CRR faithfully reflects the capability of the target LLMs trained by different methods. In Table 5, we observe that number of ties dominates the results and the number of wins are scarce. We attribute it to the fact that the target model is essentially distilling knowledge from the strong model. As a result, most of the time, the instruction-tuned target model is only able to respond as good as the strong model, through the lens of the LLM-based evaluator.

Table 4: Additional results on standard benchmarks.

| Methods | BBH | MMLU | Average |
|---|---|---|---|
| LLaMA-7B | 30.93 | 35.17 | 33.05 |
| Alpagasus | 31.55 | 36.46 | 34.01 |
| WizardLM+ | 31.72 | 37.89 | 34.81 |
| CodecLM (ours) | **32.60** | **42.67** | **37.64** |

### A.6 Consistency between LLM-based Evaluators

In the main paper, we use ChatGPT as the LLM judge for final evaluation, for its efficiency, price and accessibility for the community to reproduce our results. As pointed out in (Chiang et al., 2023), LLMs evaluators, although largely consistent with human preferences, may have their own biases. Therefore, to make sure our experimental results are solid, we also use GPT-4 as the judge and compare against the performance gap in CRR between different baselines and the Self-Instruct method. The comparison results in Table 6 demonstrates the agreement of two LLM-based judges and confirms the superior performance of CodecLM against comparing methods.

### A.7 Additional Benchmark Results

To complement the performance result using LLM-based automatic evaluator, we also evaluate LLMs tuned with the top methods presented in Section 5.4 on standard NLP benchmarks, MMLU (Hendrycks et al., 2020) and BBH (Suzgun et al., 2022). We follow the same settings introduced in (Wang et al., 2023) without demonstrations or CoT (Wei et al., 2022) prompt for evaluating the target models based on LLaMA-7B. For our method, we follow the same setting as in Evol-Instruction benchmark evaluation. We present the evaluation results in Table 4 and use the performance of vanilla LLaMA-7B as a reference. We observe the same performance ranking of all methods as that in Table 1 where we use LLM-based automatic evaluator. The consistency between two different evaluation approaches indicates the reliability of LLM-based evaluator in terms of demonstrating relative perfor-

Table 5: Detailed comparison results with LLaMA-based models on Evol-Instruct benchmark. Each method trains a target model based on LLaMA-7B or -13B, and compares against the strong model, Gemini-Pro. Capacity Recovery Ratio (%), $\texttt{CRR} = \frac{\texttt{wins+ties}}{\texttt{total comparisons}}$.

| Methods | LLaMA-7B vs. Gemini-Pro | | | | LLaMA-13B vs. Gemini-Pro | | | |
|---|---|---|---|---|---|---|---|---|
| | **Wins** | **Ties** | **Losses** | **CRR** | **Wins** | **Ties** | **Losses** | **CRR** |
| Self-Instruct | 17 | 140 | 61 | 72.02 | 29 | 136 | 53 | 75.69 |
| Alpagasus | 17 | 147 | 54 | 75.23 | 26 | 148 | 44 | 79.82 |
| Tree-Instruct | 23 | 141 | 54 | 75.23 | 26 | 154 | 38 | 82.57 |
| WizardLM | 19 | 143 | 56 | 74.31 | 30 | 149 | 39 | 82.11 |
| WizardLM+ | 19 | 146 | 53 | 75.69 | 31 | 153 | 34 | 84.40 |
| CodecLM (ours) | **29** | **145** | **44** | **79.82** | **35** | **154** | **29** | **86.70** |

Table 6: Performance gap to Self-Instruct in terms of CRR on Evol-Instruct, evaluated by ChatGPT and GPT4, respectively. Each method trains a target model based on LLaMA-7B or -13B, and compares against the strong model, Gemini-Pro. We observe two LLM-based automatic evaluators yields consistent results.

| Methods | LLaMA-7B vs. Gemini-Pro | | LLaMA-13B vs. Gemini-Pro | |
|---|---|---|---|---|
| | **ChatGPT** | **GPT4** | **ChatGPT** | **GPT4** |
| Self-Instruct | 0.00 | 0.00 | 0.00 | 0.00 |
| Alpagasus | +3.21 | +1.38 | +4.13 | +1.83 |
| Tree-Instruct | +3.21 | +2.29 | +6.88 | +4.59 |
| WizardLM | +2.29 | +0.46 | +6.42 | +3.21 |
| WizardLM+ | +3.67 | +2.29 | +8.72 | +5.50 |
| CodecLM (ours) | **+7.80** | **+8.26** | **+11.01** | **+8.72** |

mance of competing methods.

### A.8 Case Study

We present a case study in Figure 6 to show an iterative tailoring process from instruction metadata to the final high-quality prompt. In practice, the iteration may terminate earlier by the Contrastive Filtering process. We observe that Self-Rubrics is able to tailor rubrics and actions according to the given metadata. Interestingly, the actions generated by LLM seems very domain-specific. For example, the *SWOT analysis* in the last action may even be hard for non-expert human annotators to come up with. Moreover, the colored texts in instructions demonstrate that LLM is able to follow the actions quite precisely to refine the instructions.

### A.9 Prompt Templates for CodecLM

We present all prompt templates here in the appendix for better reproducibility. In particular, we list the correspondence between prompt templates and their usages as follows for quick reference:

- Figure 7: Encoding instructions into metadata, including use case and transferable skills.
- Figure 8: Decoding instruction metadata into basic instructions that are relatively simple in structure.
- Figure 9: Generating rubrics to judge how challenging an instruction is, and actions to improve the instruction based on the given metadata.
- Figure 10: Improving the input instruction by following one of the generated actions.
- Figure 11: Comparing the responses quality from the target and strong LLMs. Adapted from the Vicuna-style pairwise comparison prompt by removing the explanation part.
- Figure 12: Automatic evaluation using LLM (*e.g.*, ChatGPT, GPT-4) as the judge. Following the templates in (Chiang et al., 2023; Chen et al., 2023b)

All prompts are zero-shot except for the first encoding prompt in Figure 7, which utilizes few-shot demonstrations to showcase the LLM a rough granularity of the task and skills. Also, we choose these prompts as they work quite well in practice. And we believe recent prompt optimization techniques (Fernando et al., 2023; Yang et al., 2023) can be incorporated seamlessly into our framework, and we leave them as future work.

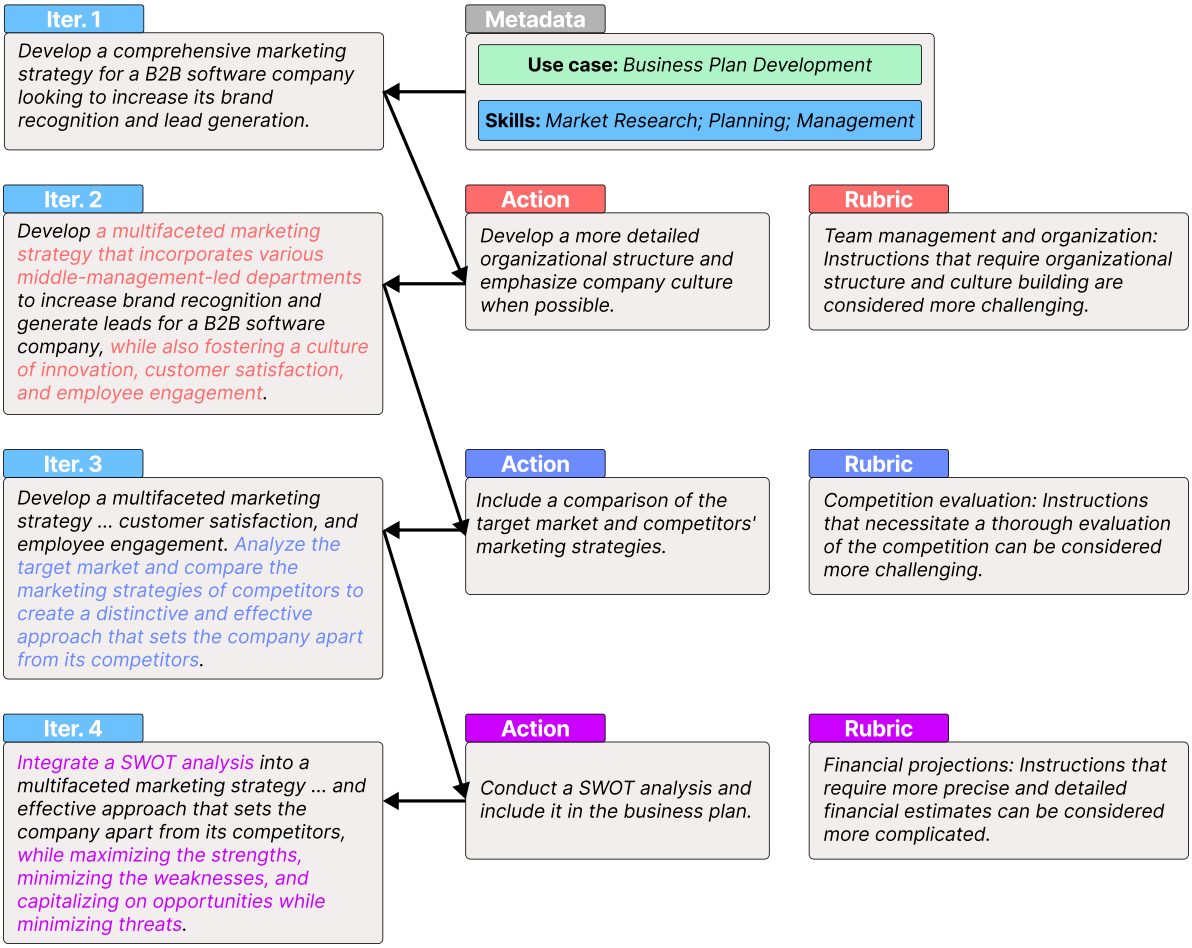


Figure 6: Case study on the instruction improvement process of CodecLM. Repetitive instructions are omitted to save space.

```
I want you to act as an instruction analyzer.
Given an instruction, you should recognize its use case and the skills (or knowledge)
required for a large language model (LLM) to answer the question.
Generate the use case and skills required without any explanation.
List at most 3 skills, each skill should be transferable, so that LLM can leverage them to answer
similar questions.
Avoid using "skill", "knowledge" to describe a skill, and each skill should be concise (2-3 words).
Follow the examples below to analyze the given instruction.

#Example 1#
As a sports commentator, describe the winning play in the final seconds of a championship game.
Use case: creative writing
Skills: role-play, sports

#Example 2#
How to read a large file (> 2T) using python?
Task: code generation
Skills: python

#Example 3#
The method section of your paper is too brief and does not explain how your proposed model works
in detail. How can you provide more details of the hierarchical encoder and the cascaded selectors,
such as their architectures, inputs, outputs, and parameters?
Task: general knowledge question answering
Skills: academic writing, machine learning

<input instruction>
<output metadata>
```

Figure 7: Prompt template to encode the input into metadata, consisting of its use case and transferable skills.

```
I want you to act as an instruction writer.
Your objective is to write <number of instructions> instructions that must be reasonable
and must be understood and responded by humans.
The generated instructions should be diverse enough while following the constraints below:

Use case of the instructions: <use case>
Skills required to respond to the instructions: <skills>

Generate the instructions without answering in numbered bulletin points.

<output instructions>
```

Figure 8: Prompt template to generate instructions from metadata.

```
I want you to act as a instruction judge with domain expertise.
Your job is to generate <number_of_rubrics> domain specific rubrics to assess the difficulty and
complexity based on the use case of the instruction, and skills required to respond to it.
The generated rubrics should be clear, concise and unambiguous.
Based on the generated rubrics, generate corresponding actions to improve an instruction by
making it more challenging.

The use case of the instruction: <use case>.
The skills required to solve the instruction: <skills>.

Generate the domain-specific rubrics and actions without explanation in numbered bulletin points:

<output rubrics>
<output actions>
```

Figure 9: Prompt template to generate actions to improve instructions based on instruction metadata.

```
I want you to act as a instruction improver with domain expertise.
Your job is to make the given instruction more challenging following the given improving action
item, and the generated instruction should be reasonable and self-consistent.
Do not directly copy words or phrases in the action.

Improving action: <action>
Input instruction: <input instruction>

Improved instruction: <output instruction>
```

Figure 10: Prompt template to improve instructions following generated actions.

```
You are a helpful and precise assistant for checking the quality of the answer.

<Question>
[The Start of Assistant 1's Answer]
<answer_1>
[The End of Assistant 1's Answer]
[The Start of Assistant 2's Answer]
<answer_2>
[The End of Assistant 2's Answer]

We would like to request your feedback on the performance of two AI assistants in response to
the user question displayed above.
Please rate the helpfulness, relevance, accuracy, level of details of their responses. Each
assistant receives an overall score on a scale of 1 to 10, where a higher score indicates
better overall performance.
Please only output a single line containing only two values indicating the scores for Assistant 1
and 2, respectively. The two scores are separated by a space.
Please avoiding any potential bias and ensuring that the order in which the responses were
presented does not affect your judgment.
```

Figure 11: Prompt template used in Contrastive Filtering to compare the responses of the strong and the target LLMs. We directly use the strong LLM with this template as the scorer $S$ to avoid additional costs from calling a third-party LLM.

```
System: You are a helpful and precise assistant for checking the quality of the answer.

User:
<Question>
[The Start of Assistant 1's Answer]
<answer_1>
[The End of Assistant 1's Answer]
[The Start of Assistant 2's Answer]
<answer_2>
[The End of Assistant 2's Answer]

We would like to request your feedback on the performance of two AI assistants in response to
the user question displayed above.
Please rate the helpfulness, relevance, accuracy, level of details of their responses. Each
assistant receives an overall score on a scale of 1 to 10, where a higher score indicates
better overall performance.
Please first output a single line containing only two values indicating the scores for Assistant 1
and 2, respectively.
The two scores are separated by a space. In the subsequent line, please provide a comprehensive
explanation of your evaluation, avoiding any potential bias and ensuring that the order in which
the responses were presented does not affect your judgment.
```

Figure 12: Prompt template for automatic evaluation using LLM (*e.g.*, ChatGPT, GPT-4) as the judge.